List of Experiments

Total No of Labs: 14

S. No.	Programs List	No. of Labs
CAD Experiments		
1	Study of basic graphics functions defined in “graphics.h”.	1
2	To implement Digital differential analyzer (DDA) Algorithm for drawing a line segment between two given end points A (x1, y1) and B(x2, y2).	1
3	To implement Bresenham’s line drawing algorithm for drawing a line segment between two given endpoints A (x1, y2) and B(x2, y2).	1
4	To implement Bresenham’s Cicrcle drawing algorithm for drawing a circle with a given center of circle P(x, y) and radius r.	1
5	To apply the basic 2D transformations such as translation, Scaling, Rotation, shearing and reflection for a given 2D object.	2
6	To apply the basic transformations such as translation, Scaling, Rotation for a given 3D object.	2
7	To generate a smooth curve by using Bezier curve technique for a given set of control points.	1
8	To implement Cohen–Sutherland 2D clipping and window–viewport mapping	1
Product Design		
9	Introduction to CAD software’s.	1
10	Generation of any one simple solid model using any CAD software.	1
11	Generation of any one assembly of mechanical part using any CAD software.	1
12	Design and develop the complete lifecycle for any product.	1

Experiment - 7

Aim: To generate a smooth curve by using Bezier curve technique for a given set of control points.

Description:

A Bézier curve is a parametric curve frequently used in computer graphics and related fields. Generalizations of Bézier curves to higher dimensions are called Bézier surfaces, of which the Béziertriangle is a special case.

In vector graphics, Bézier curves are used to model smooth curves that can be scaled indefinitely. "Paths," as they are commonly referred to in image manipulation programs are combinations of linked Bézier curves. Paths are not bound by the limits of rasterized images and are intuitive to modify. Bézier curves are also used in animation as a tool to control motion

Four points $\mathbf{P}_0$, $\mathbf{P}_1$, $\mathbf{P}_2$ and $\mathbf{P}_3$ in the plane or in higher-dimensional space define a cubic Bézier curve. The curve starts at $\mathbf{P}_0$ going toward $\mathbf{P}_1$ and arrives at $\mathbf{P}_3$ coming from the direction of $\mathbf{P}_2$. Usually, it will not pass through $\mathbf{P}_1$ or $\mathbf{P}_2$; these points are only there to provide directional information. The distance between $\mathbf{P}_0$ and $\mathbf{P}_1$ determines "how long" the curve moves into direction $\mathbf{P}_2$ before turning towards $\mathbf{P}_3$.

The explicit form of the curve is:

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3 \ , \ t \in [0,1].$$

Program for Bezier curve:

```
#include <stdio.h>
#include <stdlib.h>
#include <graphics.h>
#include <math.h>
void bezier (int x[4], int y[4])
{
int gd = DETECT, gm;
int i;
double t;
initgraph (&gd, &gm, "C:\\TurboC3\\BGI");
for (t = 0.0; t < 1.0; t += 0.0005)
{
double xt = pow (1-t, 3) * x[0] + 3 * t * pow (1-t, 2) * x[1] +
3 * pow (t, 2) * (1-t) * x[2] + pow (t, 3) * x[3];
double yt = pow (1-t, 3) * y[0] + 3 * t * pow (1-t, 2) * y[1] +
3 * pow (t, 2) * (1-t) * y[2] + pow (t, 3) * y[3];
putpixel (xt, yt, WHITE);
}
for (i=0; i<4; i++)
putpixel (x[i], y[i], YELLOW);
getch();
closegraph();
```

```
return;
}
void main()
{
int x[4], y[4];
int i;
printf ("Enter the x- and y-coordinates of the four control points.\n");
for (i=0; i<4; i++)
scanf ("%d%d", &x[i], &y[i]);
bezier (x, y);
}
```